

Elaborato per il corso di Basi di Dati
A.A. 2025/2026

CareerZen

Progetto di una base di dati per il monitoraggio e l'analisi delle
candidature lavorative

Elena Lungu
elena.lungu2@studio.unibo.it
Matricola: 0001161120

19 giugno 2026

Indice

Introduzione	3
1 Analisi dei requisiti	4
1.1 Intervista	4
1.2 Estrazione dei concetti principali	5
2 Progettazione concettuale	8
2.1 Schema Scheletro Generale	8
2.2 Analisi dei Raffinamenti e Scelte Progettuali	9
2.3 Scomposizione in Sotto-schemi	9
2.3.1 Area Utenza e Sicurezza	9
2.3.2 Area Candidature e Storicizzazione	10
2.3.3 Area Matching e Competenze	11
2.3.4 Area Iter di Selezione e Trattative	12
2.4 Schema Concettuale Finale	13
3 Progettazione logica	14
3.1 Stima del volume dei dati	14
3.2 Descrizione delle operazioni principali e stima della loro frequenza	15
3.3 Schemi di navigazione e tabelle degli accessi	16
3.4 Raffinamento dello schema	25
3.5 Traduzione di entità e associazioni in relazioni	26
3.6 Schema relazionale finale	26
3.7 Schema relazionale visivo	28
4 Implementazione Fisica	29
4.1 Definizione dello Schema (DDL)	29
4.2 Traduzione delle operazioni in query SQL (DML)	31
5 Realizzazione dell'Applicazione	36
5.1 Architettura di Sistema e Pattern DAO	36
5.2 Sicurezza, Transazioni e Integrità	36
5.3 Interfaccia Utente (GUI) e Scelte di Design	36
5.3.1 Accesso e Controllo dei Ruoli	37
5.3.2 Evoluzione in Tracker Collaborativo: Dashboard Candidato	37
5.3.3 Pannello di Controllo dell'Amministratore	38
5.4 Sviluppi Futuri	39

Introduzione

La transizione dal mondo accademico al mercato del lavoro rappresenta una delle fasi più complesse e frammentate per un profilo junior. Oggigiorno un candidato si trova spesso a dover gestire parallelamente decine di iter di selezione su piattaforme diverse, affrontando molteplici round di colloqui di diverse tipologie, scadenze disallineate e, non di rado, la totale assenza di feedback.

Ho avuto modo di toccare con mano questa realtà durante la ricerca di un'internship: un'esperienza formativa di per sé bellissima, ma al contempo un vero e proprio incubo logistico. In un panorama lavorativo estremamente saturo di giovani emergenti, gestire colloqui, test tecnici e comunicazioni frammentate si è rivelato un processo incredibilmente intenso e dispersivo.

Da questa reale esigenza nasce l'idea di **CareerZen**, una base di dati progettata specificamente per centralizzare e razionalizzare il tracciamento delle candidature lavorative. L'obiettivo del progetto non è solo quello di fornire un archivio anagrafico, ma di creare uno strumento analitico capace di storicizzare l'intero *iter* di selezione: dall'invio del curriculum, passando per i vari step di colloquio, fino all'eventuale fase di negoziazione dell'offerta economica.

1 Analisi dei requisiti

Si vuole realizzare un database a supporto della gestione e del tracciamento delle candidature lavorative per la piattaforma CareerZen. Pertanto, la base di dati dovrà gestire informazioni relative agli utenti, alle aziende, alle posizioni lavorative aperte, al relativo iter di selezione, stati di avanzamento, colloqui e offerte economiche.

I candidati potranno monitorare l'evoluzione delle proprie sottomissioni, registrare i feedback per individuare lacune e valutare la compatibilità delle proprie competenze con i requisiti richiesti, mentre gli amministratori si occuperanno della gestione centralizzata delle anagrafiche.

1.1 Intervista

Un primo testo ottenuto dall'intervista è il seguente:

La startup CareerZen richiede un sistema informatico di supporto per la gestione e il tracciamento delle candidature lavorative, ideato per ridurre lo stress dei candidati durante l'iter di selezione. Il database di informazioni verrà acceduto principalmente dagli utenti in cerca di impiego, ma prevede anche un lato amministrativo per la gestione delle anagrafiche standard.

Il sistema deve poter memorizzare la lista degli utenti iscritti, ciascuno dei quali avrà un proprio account al quale sarà associato un indirizzo email, una password, nome e cognome. Il candidato ha la possibilità di arricchire il proprio profilo inserendo le proprie competenze tecniche (skill), indicando per ciascuna di esse un livello di padronanza tramite un punteggio numerico da 1 a 5. Il sistema mantiene un'anagrafica delle aziende presso cui è possibile candidarsi, memorizzandone il nome, il settore di riferimento e la sede principale.

Ogni azienda può avere aperte diverse posizioni lavorative. Per ogni posizione è necessario specificare il titolo del ruolo, la modalità di lavoro (remoto o in sede), la data di scadenza dell'annuncio e il range di RAL (Retribuzione Annua Lorda) previsto. Inoltre, ogni posizione lavorativa richiede specifiche skill, anch'esse associate a un livello minimo richiesto, per permettere al sistema di valutare la compatibilità con il candidato.

Quando un utente decide di applicare per un annuncio, il sistema genera una candidatura. Di questa candidatura è fondamentale tracciare l'intero storico degli stati di avanzamento (ad esempio: inviata, in revisione, rifiutata, ritirata), registrando la data di ogni cambio di stato. Il processo di selezione prevede spesso dei round di colloquio. Per ogni colloquio associato a una candidatura, il sistema memorizza la data, il tipo di incontro (conoscitivo, tecnico, HR, finale), il nome del recruiter di riferimento e un campo di testo libero in cui l'utente può inserire le proprie note e l'esito dello step.

Se l'iter va a buon fine, l'azienda propone un'offerta economica. Poiché le trattative possono includere rilanci e contro-proposte, il sistema deve storicizzare tutte le offerte ricevute per una singola candidatura, registrando la RAL proposta, eventuali benefit, la data di validità e lo stato finale dell'offerta (accettata o rifiutata dall'utente). Il sistema permetterà infine di generare statistiche, come il tempo medio di risposta delle aziende o la percentuale di conversione dei colloqui in offerte.

1.2 Estrazione dei concetti principali

Per garantire una transizione fluida verso la progettazione concettuale, i termini chiave emersi durante l'intervista vengono strutturati e classificati nella tabella del glossario.

Termine	Categoria	Descrizione	Note / Sinonimi
Utente	Entità	Persona registrata sulla piattaforma per monitorare le selezioni.	Sinonimo: Candidato. Identificato dalla mail.
Azienda	Entità	Società o organizzazione presso cui l'utente si candida.	Memorizza nome, settore e sede principale.
Posizione	Entità	Annuncio di lavoro o stage aperto da un'azienda.	Contiene titolo, modalità, RAL e scadenza.
Candidatura	Entità	Atto dell'utente di applicare per una determinata posizione.	Fa da ponte logico tra Utente e Posizione.
Storico Stati	Ent. Debole	Timeline e datazione di ogni cambio di stato della candidatura.	Indispensabile per calcolare i tempi medi.
Colloquio	Entità	Singolo round o step dell'iter di selezione (tecnico, HR).	Una candidatura prevede più colloqui.
Offerta	Entità	Proposta economica e contrattuale avanzata dall'azienda.	Storicizza anche eventuali rilanci economici.
Skill	Entità	Competenza tecnica presente nel sistema (es. Java, SQL).	Gestita tramite un'anagrafica centralizzata.
Livello	Attributo	Valutazione numerica (1-5) posta sulle relazioni.	Attributo esclusivo delle associazioni <i>Competenza</i> e <i>Requisito</i> .

Occorre fare alcune precisazioni operative per evitare ambiguità nella modellazione:

- Il termine **Ruolo** all'interno del sistema fa riferimento esclusivamente ai livelli di accesso e ai privilegi dell'account (es. *Utente Base* vs *Amministratore*). L'eventuale specializzazione professionale del candidato (es. *Sviluppatore Web*, *Data Analyst*) non è modellata come un'etichetta statica, ma viene derivata dinamicamente dall'insieme delle **Skill** tecniche possedute e dal relativo livello di padronanza.
- Il "Recruiter" non rappresenta un'entità a sé stante con accesso al sistema, ma costituisce un semplice attributo testuale all'interno dell'entità *Colloquio*.
- Le "Skill" non sono campi di testo libero, ma istanze di un'anagrafica globale; questo vincolo è essenziale per calcolare il livello di compatibilità (match) tra le competenze dell'utente e i requisiti della posizione.

Specifiche ristrutturate

A seguito della strutturazione del glossario e della risoluzione delle ambiguità lessicali, si procede alla stesura di un testo ristrutturato che riassume le specifiche in modo rigoroso:

Il sistema gestisce Utenti (identificati da email, con nome, cognome e password) e Aziende (identificate da un ID univoco, con nome, settore e sede). Un Utente può possedere diverse Skill, valutate con un Livello da 1 a 5. Le Aziende pubblicano Posizioni Lavorative, caratterizzate da titolo, modalità, RAL, scadenza e da una serie di Skill richieste come requisito (anch'esse con un Livello minimo).

L'Utente può inviare una Candidatura per una Posizione. Di ogni Candidatura si traccia l'avanzamento generando un'entità debole Storico Stati (con data e stato). L'iter prevede la creazione di zero o più Colloqui (con data, tipo, recruiter e note opzionali) e può culminare nella ricezione di una o più Offerte (con RAL proposta, benefit, scadenza e stato di accettazione), storicizzate per gestire i rilanci. Esiste infine la figura dell'Amministratore, un particolare Utente con privilegi di gestione sull'anagrafica globale.

Elenco delle principali azioni

Segue un elenco delle principali azioni richieste dal sistema, che costituiranno il carico di lavoro del database:

1. Iscrivere un nuovo Utente (Candidato o Amministratore).
2. Registrare una nuova Azienda nel sistema.
3. Pubblicare una nuova Posizione Lavorativa con le relative Skill richieste.
4. Associare una nuova Skill al profilo di un Utente, specificandone il Livello.
5. Registrare l'invio di una nuova Candidatura da parte di un Utente.
6. Inserire un nuovo step di Colloquio per una Candidatura esistente.
7. Registrare un cambio di stato per una Candidatura (storicizzazione nell'entità debole).
8. Memorizzare una nuova Offerta economica ricevuta.
9. Visualizzare lo storico completo degli stati, dei colloqui e delle offerte di una specifica Candidatura.
10. Ricercare Posizioni Lavorative aperte filtrando per compatibilità con le Skill dell'Utente.
11. Segnalare e bloccare un'Azienda (operazione riservata all'Amministratore).
12. Aggiornare l'esito e le note di un Colloquio precedentemente registrato.
13. Registrare la decisione finale (accettazione o rifiuto) da parte del candidato su un'Offerta economica.
14. Visualizzare la dashboard principale del Candidato, includendo l'elenco e lo stato corrente delle sue candidature attive.

15. Generare un report statistico sulle performance aziendali (es. tempo medio di risposta e tasso di conversione in offerte).
16. Modificare il livello di padronanza di una Skill già associata al profilo di un Utente.

2 Progettazione concettuale

2.1 Schema Scheletro Generale

In questa prima fase di modellazione concettuale viene delineata l'ossatura globale della base di dati. L'attenzione è focalizzata unicamente sulle entità fondamentali e sulle relazioni bidirezionali che le connettono, omettendo temporaneamente la specifica degli attributi descrittivi per analizzare la pura logica delle cardinalità di sistema.

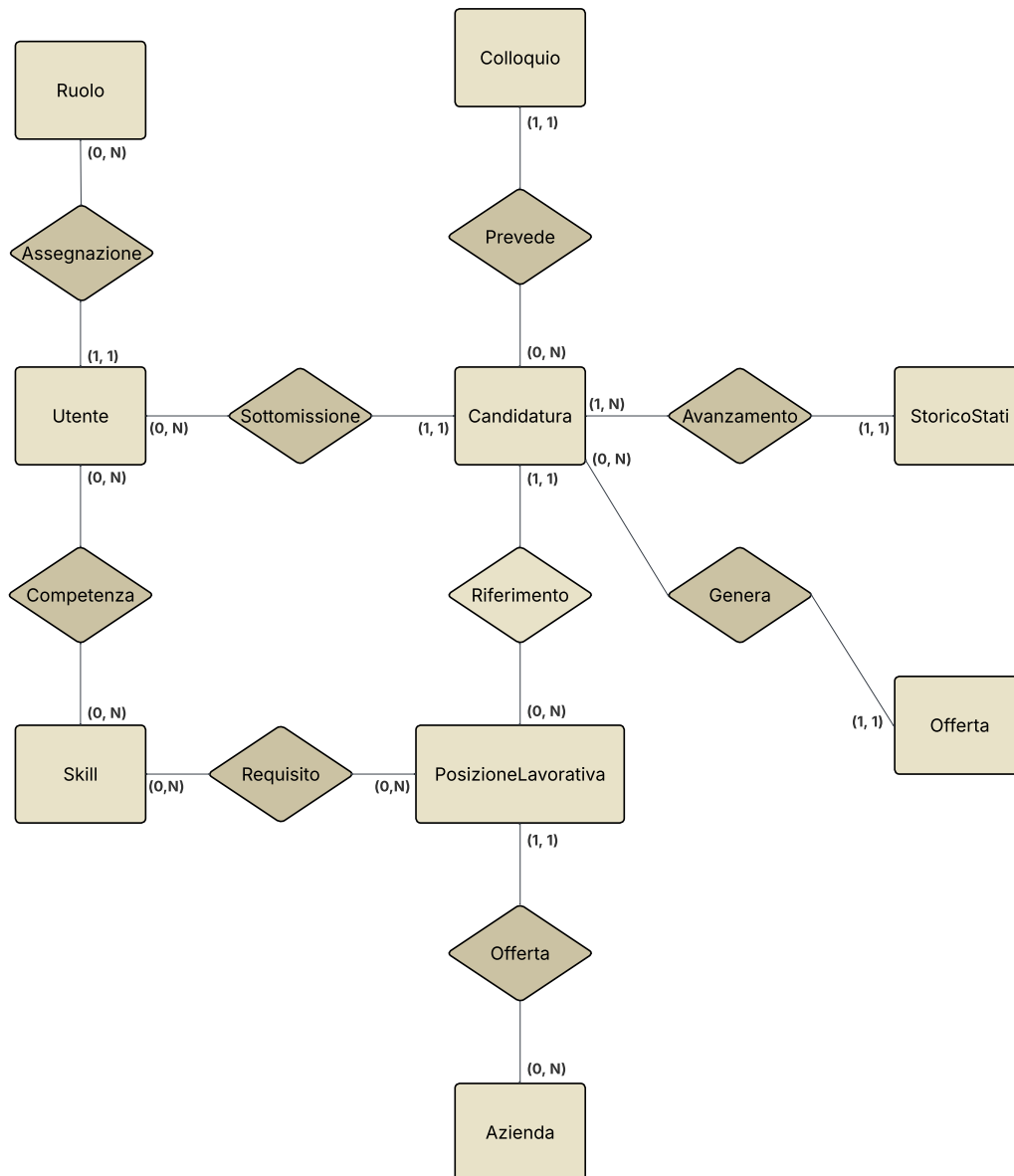


Figura 1: Schema Scheletro generale dell'applicazione CareerZen.

2.2 Analisi dei Raffinamenti e Scelte Progettuali

Prima di procedere alla scomposizione in sotto-schemi e alla stesura del diagramma definitivo, l'ossatura iniziale è stata sottoposta a un processo di ottimizzazione semantica per eliminare le ambiguità del linguaggio naturale.

I principali interventi hanno riguardato la rinormalizzazione delle associazioni tramite sostantivi neutrali (es. *Sottomissione*, *Competenza*) per garantire la perfetta bidirezionalità del modello, l'introduzione di cardinalità parziali (0, 1) per gestire nativamente i dati opzionali (evitando record fittizi o stringhe vuote), e l'isolamento dei vincoli temporali tramite entità deboli per impedire sovrascritture distruttive dei dati storici.

2.3 Scomposizione in Sotto-schemi

Per analizzare nel dettaglio le regole del dominio applicativo, lo schema concettuale è stato scomposto in quattro macro-aree di interesse indipendenti.

2.3.1 Area Utenza e Sicurezza

L'architettura dei dati centralizza i dati anagrafici e le credenziali di accesso (email e password) nella superclasse *Utente*, azzerando le ridondanze nel database. La netta separazione tra i due soggetti che interagiscono sulla piattaforma ha poi richiesto l'introduzione di una generalizzazione totale ed esclusiva (**T, E**) nelle due sottoclassi *Candidato* e *Amministratore*. Isolare i due profili risponde a un preciso vincolo logico: l'invio di una candidatura o l'associazione delle skill sono prerogative esclusive del *Candidato*. L'*Amministratore* deve limitarsi alle sole funzioni di controllo e moderazione globale dell'anagrafica. Questa scomposizione impedisce a monte, direttamente tramite i vincoli di integrità del modello, che un profilo con privilegi amministrativi possa generare record anomali, come il candidarsi a una posizione lavorativa.

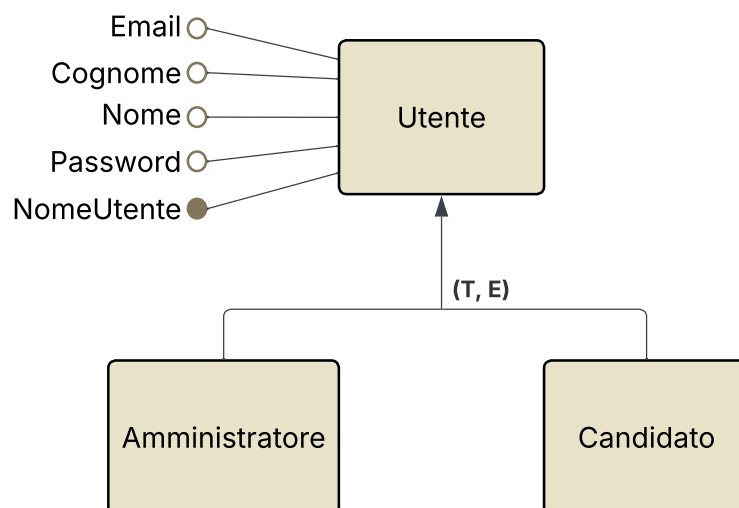


Figura 2: Sotto-schema concettuale: Area Utenza e Sicurezza.

2.3.2 Area Candidature e Storicizzazione

Il blocco logico dedicato alle candidature costituisce il motore dell'applicazione, in cui l'entità *Candidatura* funge da collante tra il profilo del candidato e lo specifico annuncio lavorativo.

Poiché un iter di selezione è per natura dinamico, era fondamentale non limitarsi a salvare solo lo stato corrente (ad esempio “in valutazione” o “respinta”), perdendo così traccia dei passaggi precedenti. Per risolvere il problema, abbiamo affiancato alla candidatura l'entità debole *Storico Stati*. Sfruttando il vincolo di dipendenza nell'associazione *Avanzamento*, il database non sovrascrive mai i dati: ogni volta che l'iter procede, viene generato un nuovo record con la data aggiornata, costruendo una timeline affidabile e completamente storicizzata per ogni singola sottomissione.

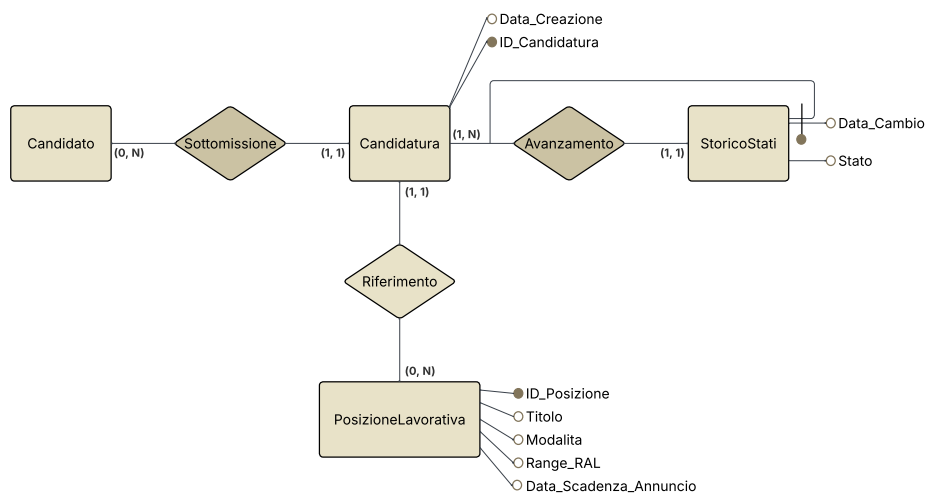


Figura 3: Sotto-schema concettuale: Area Candidature.

2.3.3 Area Matching e Competenze

Questa sezione definisce il motore di ricerca delle competenze tecniche, racchiuse nell'entità *Skill*. Il nodo progettuale più interessante era definire la gestione del punteggio (da 1 a 5) relativo all'esperienza dell'utente o ai requisiti dell'annuncio.

Invece di forzare la modellazione di questo dato direttamente sulle entità anagrafiche, la logica adottata prevede che esso viva esclusivamente nell'intersezione tra le parti. Come verrà matematicamente formalizzato e implementato durante la successiva fase di progettazione logica, gli attributi *Livello* e *Livello_Minimo* verranno agganciati direttamente sulle associazioni multi-a-molti *Competenza* e *Requisito*. La ratio è strettamente pragmatica: il livello di padronanza non ha senso di esistere in astratto, ma assume un significato reale solo nel momento esatto in cui un candidato lo dichiara, o un'azienda lo impone come filtro. Questa architettura permetterà di mantenere il database pulito ed evitare dispersioni di dati.

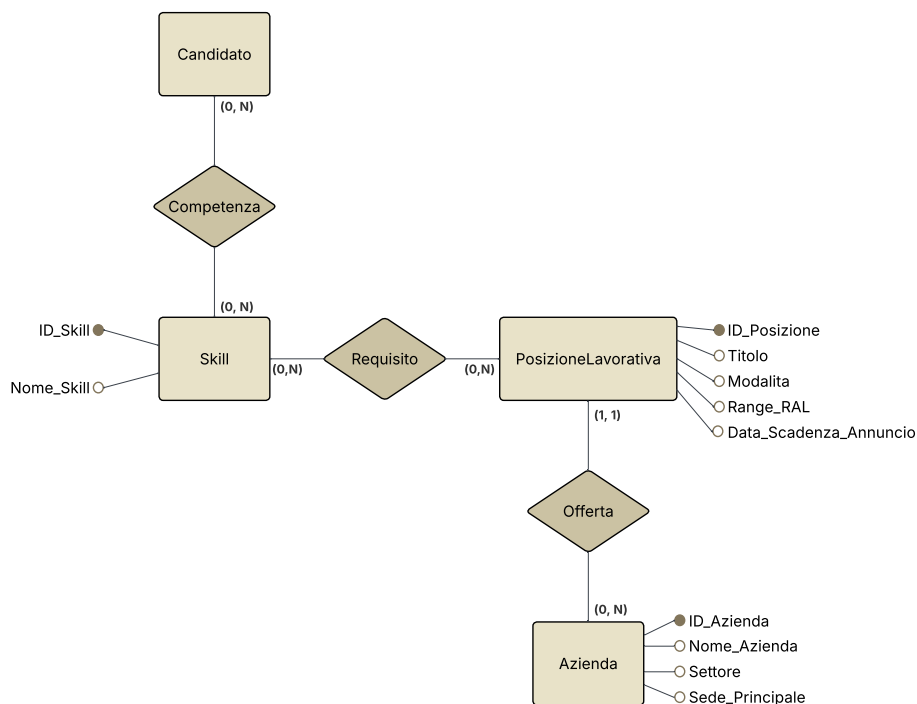


Figura 4: Sotto-schema concettuale: Area Matching delle Competenze.

2.3.4 Area Iter di Selezione e Trattative

Quest'ultima sezione copre le fasi conclusive e più delicate dell'intero percorso: lo svolgimento dei colloqui e la gestione delle proposte economiche.

L'entità *Colloquio* ha il compito di mappare ogni singolo step di selezione. Una scelta di design specifica è stata quella di rendere opzionali gli attributi come *Recruiter* e *Note*. Questo serve a gestire i casi, oggi molto frequenti, in cui i primi round consistono in test tecnici automatizzati o asincroni su piattaforme esterne, contesti in cui non è presente una figura umana di riferimento.

L'entità *Offerta* si occupa invece della fase di contrattazione. Poiché una trattativa difficilmente si chiude al primo tentativo, ma procede spesso per tentativi, rilanci sulla RAL e modifiche ai benefit, abbiamo evitato di salvare solo l'esito finale. La struttura permette di registrare ogni singola proposta avanzata dall'azienda, preservando l'intera cronologia delle trattative fino alla decisione definitiva.

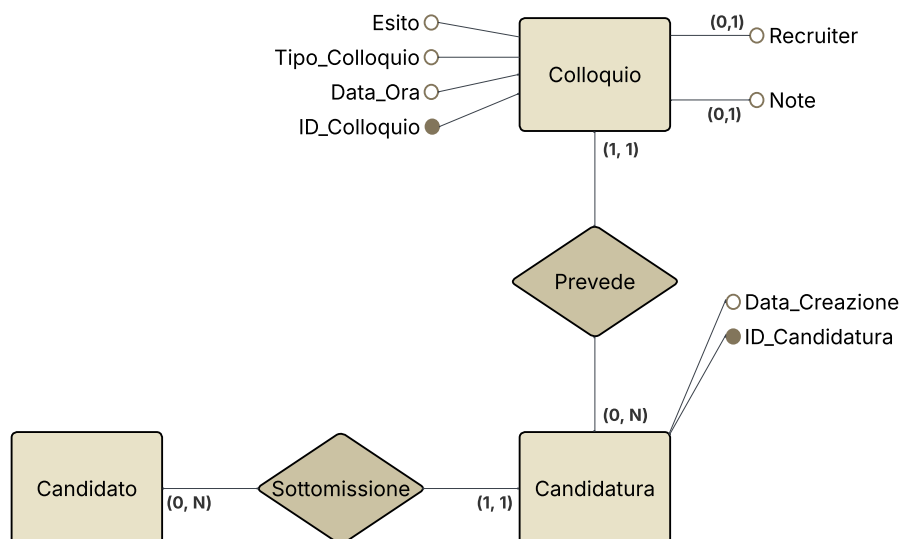


Figura 5: Sotto-schema concettuale: Area Colloqui e Offerte.

2.4 Schema Concettuale Finale

L'unione e l'integrazione dei sotto-schemi precedentemente analizzati si concretizza nel diagramma concettuale globale definitivo, comprensivo di tutte le entità, attributi identificativi, associazioni e vincoli strutturali.

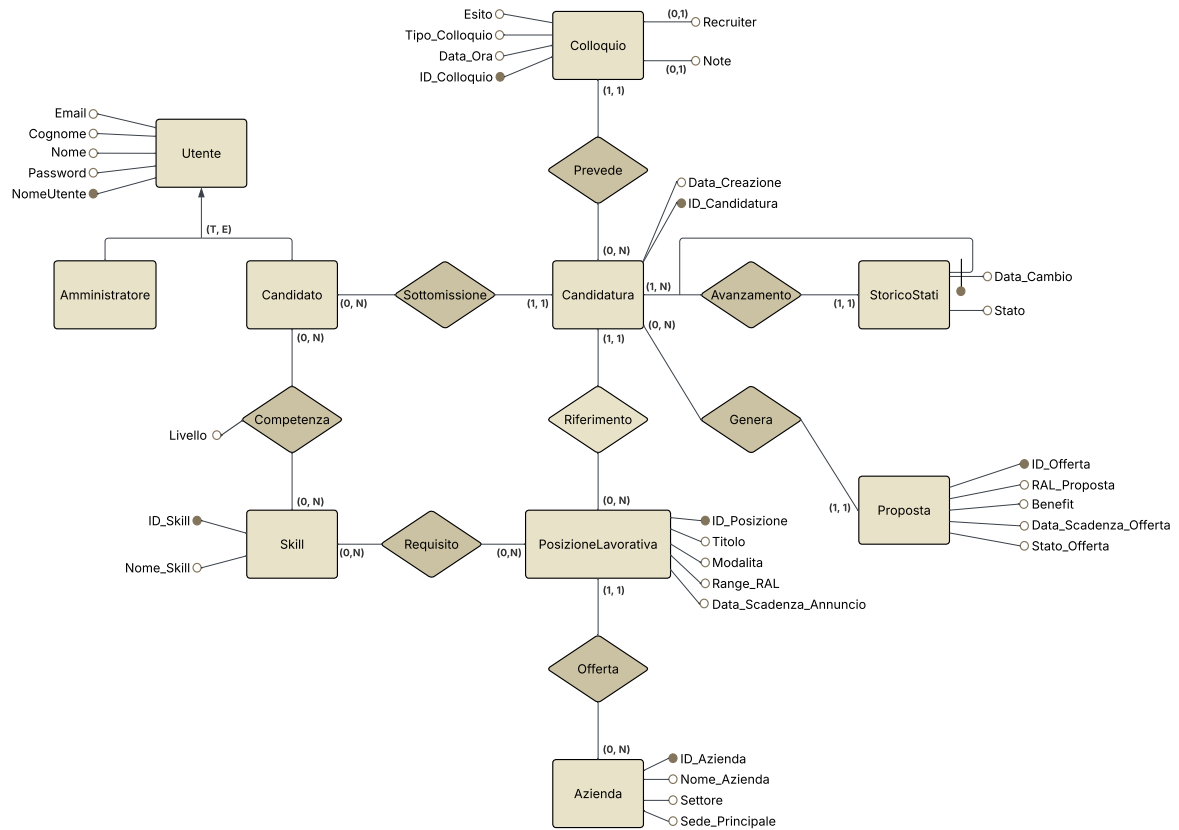


Figura 6: Schema Concettuale Finale completo di “CareerZen”.

3 Progettazione logica

3.1 Stima del volume dei dati

Concetto	Costrutto	Volume	Giustificazione
<i>Utente</i>	Entità	5.020	Somma dei profili attivi a sistema (generalizzazione totale).
<i>Candidato</i>	Entità (Sub)	5.000	Utenza base registrata per la ricerca di impiego.
<i>Amministratore</i>	Entità (Sub)	20	Staff interno per la moderazione della piattaforma.
<i>Azienda</i>	Entità	1.000	Anagrafica delle imprese partner presenti nel sistema.
<i>PosizioneLavorativa</i>	Entità	5.000	Si stimano in media 5 annunci aperti all'anno per azienda.
<i>Offerta</i>	Relazione	5.000	Associazione 1:N. Coincide con il volume di PosizioneLavorativa.
<i>Skill</i>	Entità	200	Catalogo di tecnologie, linguaggi e soft skill.
<i>Competenza</i>	Relazione	50.000	Si stimano 10 skill tecniche dichiarate per ogni candidato.
<i>Requisito</i>	Relazione	25.000	Si stimano 5 skill richieste per ogni posizione lavorativa.
<i>Candidatura</i>	Entità	125.000	Media di 25 application inviate annualmente da ciascun candidato.
<i>Sottomissione</i>	Relazione	125.000	Associazione 1:N. Coincide con il volume delle Candidature.
<i>Riferimento</i>	Relazione	125.000	Associazione 1:N. Coincide con il volume delle Candidature.
<i>StoricoStati</i>	Entità Debole	375.000	Si stimano in media 3 cambi di stato per ogni candidatura.
<i>Avanzamento</i>	Relazione	375.000	Associazione 1:N. Coincide con i record dello StoricoStati.
<i>Colloquio</i>	Entità	12.500	Il 10% delle candidature genera uno step di colloquio.
<i>Prevede</i>	Relazione	12.500	Associazione 1:N. Coincide con il volume dei Colloqui.
<i>Proposta</i>	Entità	1.250	Il 10% dei colloqui si concretizza con una proposta economica.
<i>Genera</i>	Relazione	1.250	Associazione 1:N. Coincide con il volume delle Offerte.

Tabella 1: Tabella completa dei volumi stimati (entità e relazioni) per un anno di operatività.

3.2 Descrizione delle operazioni principali e stima della loro frequenza

Per quantificare il carico di lavoro del database, le operazioni principali individuate durante l'analisi dei requisiti sono state classificate per tipologia: Interattiva (I) per le azioni utente in real-time, o Batch (B) per le elaborazioni analitiche pesanti. Le frequenze giornaliere sono state ricavate matematicamente dai volumi annuali stimati in precedenza.

ID	Operazione	Tipo	Frequenza (giornaliera)
O1	Iscrivere un nuovo Utente (Candidato o Amministratore).	I	15
O2	Registrare una nuova Azienda nel sistema.	I	3
O3	Pubblicare una nuova Posizione Lavorativa con le relative Skill.	I	15
O4	Associare una nuova Skill al profilo di un Utente (Livello).	I	140
O5	Registrare l'invio di una nuova Candidatura da parte di un Utente.	I	350
O6	Inserire un nuovo step di Colloquio per una Candidatura esistente.	I	35
O7	Registrare un cambio di stato per una Candidatura (storicizzazione).	I	1.000
O8	Memorizzare una nuova Offerta economica ricevuta.	I	4
O9	Visualizzare lo storico completo degli stati, colloqui e offerte.	I	500
O10	Ricerca Posizioni Lavorative filtrando per compatibilità (Skill).	I	3.000
O11	Segnalare e bloccare un'Azienda (riservata all'Amministratore).	I	1
O12	Aggiornare l'esito e le note di un Colloquio precedentemente registrato.	I	30
O13	Registrare la decisione finale su un'Offerta economica.	I	3
O14	Visualizzare la dashboard principale del Candidato (stato candidature).	I	2.000
O15	Generare un report statistico sulle performance aziendali.	B	50
O16	Modificare il livello di padronanza di una Skill per un Utente.	I	25

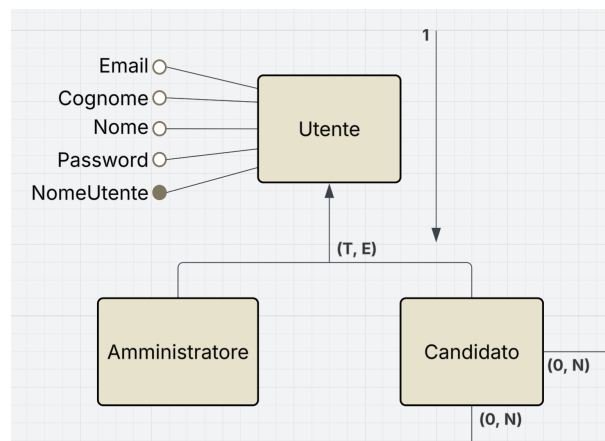
Tabella 2: Elenco completo delle operazioni e stima della frequenza giornaliera.

3.3 Schemi di navigazione e tabelle degli accessi

In questa sezione viene analizzato il costo computazionale di tutte le operazioni previste dal sistema. Per ciascuna di esse è quantificato il numero di accessi logici al database, esplicitando i costrutti visitati (entità e relazioni). Come da standard metodologico, viene applicato un peso raddoppiato alle operazioni di scrittura (1 Scrittura = 2 Letture).

O1 - Iscrivere un nuovo Utente (Candidato o Amministratore)

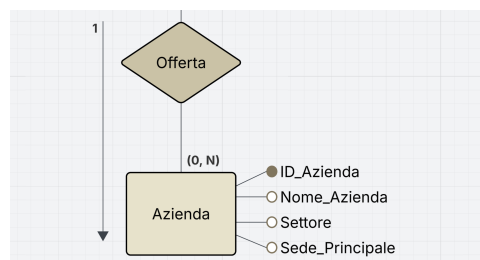
Il percorso logico parte dall'entità *Utente* e, attraversando la gerarchia, si dirama verso le entità figlie *Candidato* o *Amministratore* a seconda del ruolo scelto in fase di registrazione.



Concetto	Costrutto	Accessi	Tipo	Costo parziale
<i>Utente</i>	Entità	1	S	2
<i>Candidato</i>	Entità (Sub)	1	S	2
Totale: 4 accessi $\times$ 15/giorno = 60 accessi al giorno				

O2 - Registrare una nuova Azienda nel sistema

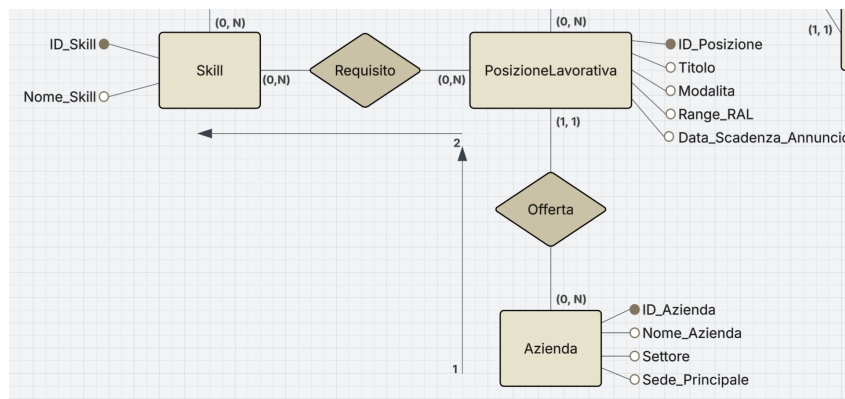
Trattandosi di un inserimento anagrafico indipendente, la navigazione si esaurisce nell'accesso in scrittura diretto all'entità *Azienda*, senza attraversare ulteriori relazioni.



Concetto	Costrutto	Accessi	Tipo	Costo parziale
<i>Azienda</i>	Entità	1	S	2
Totale: 2 accessi $\times$ 3/giorno = 6 accessi al giorno				

O3 - Pubblicare una nuova Posizione Lavorativa con le relative Skill

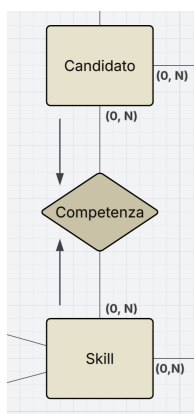
L'operazione prende in input l'entità *Azienda*, genera una nuova *PosizioneLavorativa* tramite la relazione *Proposta* e naviga infine verso *Skill* attraverso la relazione *Requisito* per fissare le competenze minime richieste.



Concetto	Costrutto	Accessi	Tipo	Costo parziale
<i>Azienda</i>	Entità	1	L	1
<i>PosizioneLavorativa</i>	Entità	1	S	2
<i>Proposta</i>	Relazione	1	S	2
<i>Skill</i>	Entità	5	L	5
<i>Requisito</i>	Relazione	5	S	10
Totale: 20 accessi × 15/giorno = 300 accessi al giorno				

O4 - Associare una nuova Skill al profilo di un Utente

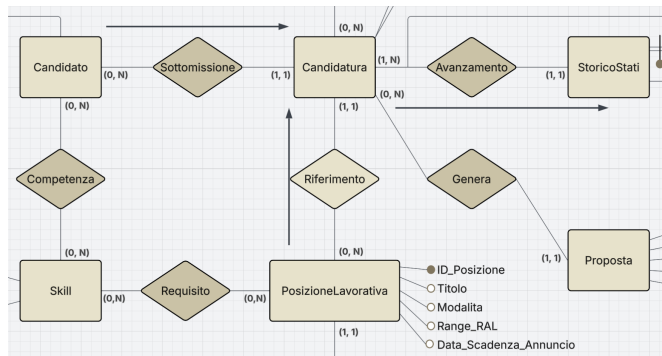
Il flusso collega l'entità *Candidato* al catalogo *Skill* istanziando un nuovo record nella relazione *Competenza*, valorizzandone contestualmente l'attributo relativo al livello.



Concetto	Costrutto	Accessi	Tipo	Costo parziale
<i>Candidato</i>	Entità	1	L	1
<i>Skill</i>	Entità	1	L	1
<i>Competenza</i>	Relazione	1	S	2
Totale: 4 accessi × 140/giorno = 560 accessi al giorno				

O5 - Registrare l'invio di una nuova Candidatura

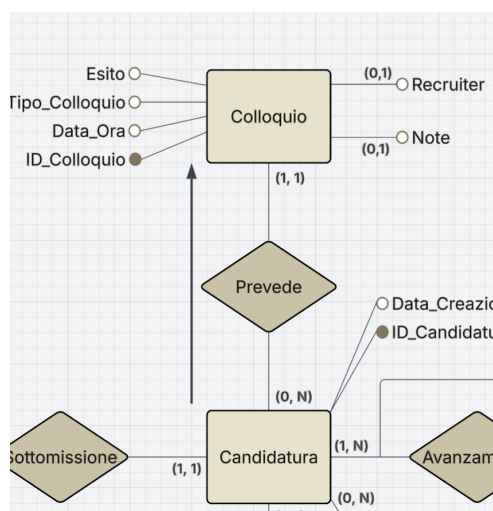
Il flusso logico prevede la lettura dei profili esistenti (*Candidato* e *PosizioneLavorativa*) per generare la nuova entità *Candidatura*. Contestualmente, l'operazione naviga verso l'entità debole *StoricoStati* per inizializzare il primo stato dell'iter contrattuale (es. "Inviata").



Concetto	Costrutto	Accessi	Tipo	Costo parziale
<i>Candidato</i>	Entità	1	L	1
<i>PosizioneLavorativa</i>	Entità	1	L	1
<i>Candidatura</i>	Entità	1	S	2
<i>Sottomissione</i>	Relazione	1	S	2
<i>Riferimento</i>	Relazione	1	S	2
<i>StoricoStati</i>	Entità Debole	1	S	2
<i>Avanzamento</i>	Relazione	1	S	2
Totale: 12 accessi $\times$ 350/giorno = 4.200 accessi al giorno				

O6 - Inserire un nuovo step di Colloquio

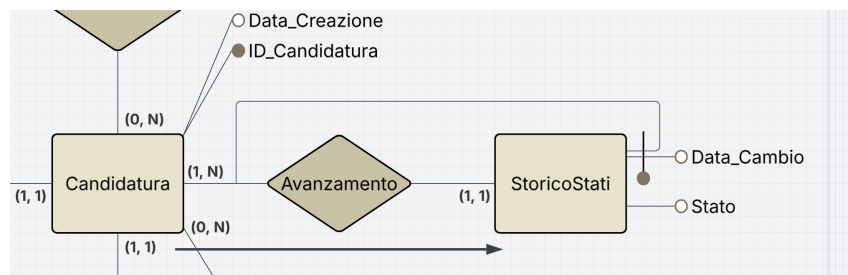
Partendo da una *Candidatura* specifica in lettura, la navigazione attraverso la relazione *Prevede* per istanziare un nuovo record nell'entità *Colloquio*, registrando i dettagli del round di selezione.



Concetto	Costrutto	Accessi	Tipo	Costo parziale
<i>Candidatura</i>	Entità	1	L	1
<i>Colloquio</i>	Entità	1	S	2
<i>Prevede</i>	Relazione	1	S	2
Totale: 5 accessi $\times$ 35/giorno = 175 accessi al giorno				

O7 - Registrare un cambio di stato per una Candidatura

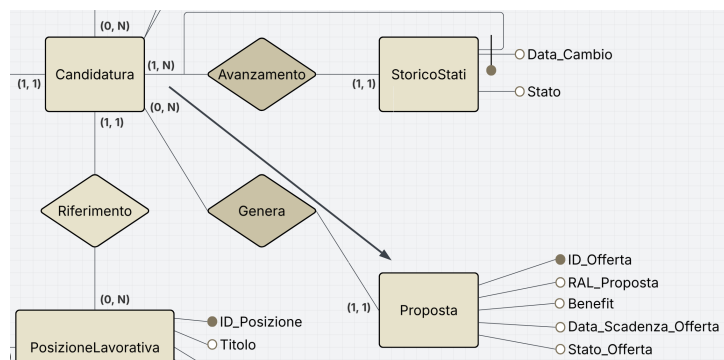
L'operazione isola una *Candidatura* attiva e naviga lungo la relazione *Avanzamento* per inserire una nuova transizione temporale nell'entità debole *StoricoStati*, aggiornando in modalità append-only la fase corrente dell'applicazione.



Concetto	Costrutto	Accessi	Tipo	Costo parziale
<i>Candidatura</i>	Entità	1	L	1
<i>StoricoStati</i>	Entità Debole	1	S	2
<i>Avanzamento</i>	Relazione	1	S	2
Totale: 5 accessi $\times$ 1.000/giorno = 5.000 accessi al giorno				

O8 - Memorizzare una nuova Offerta economica ricevuta

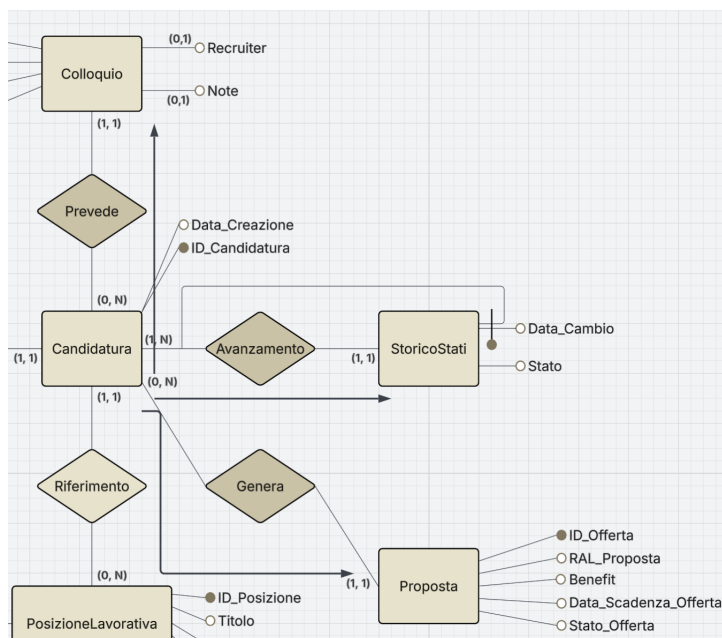
Identificata la *Candidatura* giunta al termine dell'iter selettivo, il percorso di scrittura attraversa la relazione *Genera* per storicizzare i termini contrattuali all'interno dell'entità *Offerta*.



Concetto	Costrutto	Accessi	Tipo	Costo parziale
<i>Candidatura</i>	Entità	1	L	1
<i>Offerta</i>	Entità	1	S	2
<i>Genera</i>	Relazione	1	S	2
Totale: 5 accessi $\times$ 4/giorno = 20 accessi al giorno				

O9 - Visualizzare lo storico completo di una Candidatura

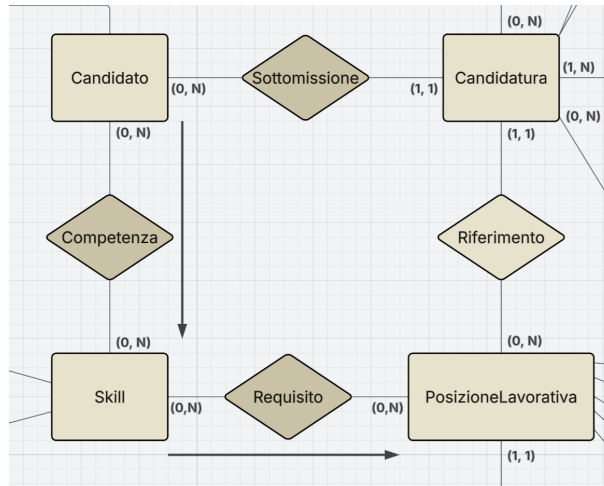
L'operazione esegue una lettura radiale partendo dal fulcro dell'entità *Candidatura*. Il flusso si espande contemporaneamente verso le tre direttrici dell'iter selettivo: la cronologia in *StoricoStati*, i round pianificati in *Colloquio* e le eventuali proposte contrattuali in *Offerta*.



Concetto	Costrutto	Accessi	Tipo	Costo parziale
<i>Candidatura</i>	Entità	1	L	1
<i>Avanzamento</i>	Relazione	3	L	3
<i>StoricoStati</i>	Entità Debole	3	L	3
<i>Prevede</i>	Relazione	1	L	1
<i>Colloquio</i>	Entità	1	L	1
<i>Genera</i>	Relazione	1	L	1
<i>Offerta</i>	Entità	1	L	1
Totale: 11 accessi × 500/giorno = 5.500 accessi al giorno				

O10 - Ricercare Posizioni Lavorative filtrando per compatibilità

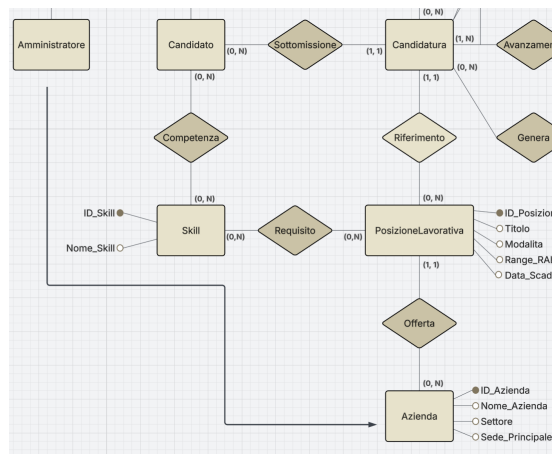
Rappresenta l'operazione più onerosa in lettura dell'intero sistema. Il flusso traccia una catena logica sequenziale: parte dal *Candidato*, ne estrae le *Skill* tramite la relazione di *Competenza* e mappa tali competenze sui requisiti di ingresso delle varie *PosizioniLavorative*.



Concetto	Costrutto	Accessi	Tipo	Costo parziale
<i>Candidato</i>	Entità	1	L	1
<i>Competenza</i>	Relazione	10	L	10
<i>Skill</i>	Entità	10	L	10
<i>Requisito</i>	Relazione	100	L	100
<i>PosizioneLavorativa</i>	Entità	20	L	20
Totale: 141 accessi $\times$ 3.000/giorno = 423.000 accessi al giorno				

O11 - Segnalare e bloccare un'Azienda

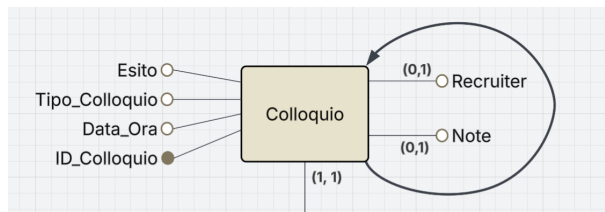
L'operazione coinvolge lo staff di moderazione. Il percorso logico si attiva in lettura sull'entità *Amministratore* per la verifica dei privilegi e si propaga con un accesso in modifica sull'anagrafica dell'entità *Azienda* per variane lo stato operativo.



Concetto	Costrutto	Accessi	Tipo	Costo parziale
<i>Amministratore</i>	Entità	1	L	1
<i>Azienda</i>	Entità	1	L	1
<i>Azienda</i>	Entità	1	S	2
Totale: 4 accessi $\times$ 1/giorno = 4 accessi al giorno				

O12 - Aggiornare l'esito e le note di un Colloquio

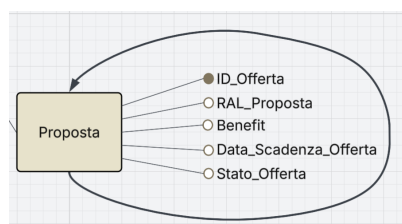
Trattandosi di una modifica puntuale a seguito dell'avvenuta valutazione di uno step selettivo, la navigazione esegue una ricerca in lettura del record specifico nell'entità *Colloquio* e ne sovrascrive i campi informativi in scrittura.



Concetto	Costrutto	Accessi	Tipo	Costo parziale
<i>Colloquio</i>	Entità	1	L	1
<i>Colloquio</i>	Entità	1	S	2
Totale: 3 accessi $\times$ 30/giorno = 90 accessi al giorno				

O13 - Registrare la decisione finale su un'Offerta economica

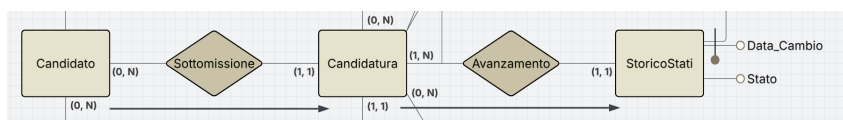
Trattandosi di un'operazione di aggiornamento su un accordo già in essere, la navigazione si esaurisce in un accesso in lettura per il recupero del record e una successiva sovrascrittura (scrittura) locale sull'entità *Offerta*, modificando l'attributo di stato.



Concetto	Costrutto	Accessi	Tipo	Costo parziale
<i>Offerta</i>	Entità	1	L	1
<i>Offerta</i>	Entità	1	S	2
Totale: 3 accessi $\times$ 3/giorno = 9 accessi al giorno				

O14 - Visualizzare la dashboard principale del Candidato

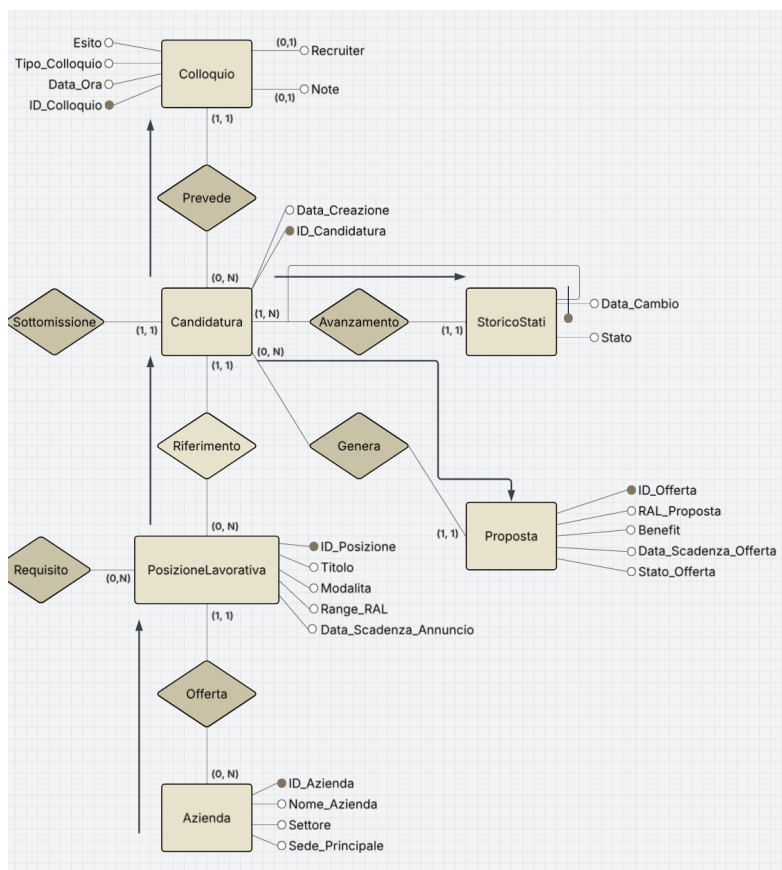
Il flusso alla base dell'interfaccia utente principale: partendo dal profilo del *Candidato*, si naviga tramite *Sottomissione* per rintracciare tutte le *Candidature* attive, interrogando infine l'entità debole *StoricoStati* per ricavarne la transizione più recente.



Concetto	Costrutto	Accessi	Tipo	Costo parziale
<i>Candidato</i>	Entità	1	L	1
<i>Sottomissione</i>	Relazione	25	L	25
<i>Candidatura</i>	Entità	25	L	25
<i>Avanzamento</i>	Relazione	75	L	75
<i>StoricoStati</i>	Entità Debole	75	L	75
Totale: 201 accessi $\times$ 2.000/giorno = 402.000 accessi al giorno				

O15 - Generare un report statistico sulle performance aziendali

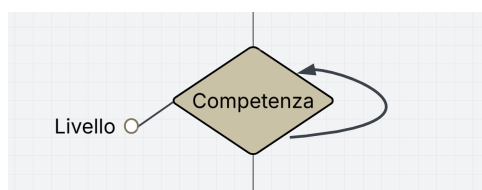
È l'elaborazione analitica (Batch) più profonda del sistema. A partire da un'istanza di *Azienda*, esplora in cascata tutte le dipendenze strutturali, attraversando gli annunci (*PosizioneLavorativa*), raccogliendo le relative *Candidature* e diramandosi radialmente su *Colloqui*, *Offerte* e cronologia degli *Stati*.



Concetto	Costrutto	Accessi	Tipo	Costo parziale
<i>Azienda</i>	Entità	1	L	1
<i>Proposta</i>	Relazione	5	L	5
<i>PosizioneLavorativa</i>	Entità	5	L	5
<i>Riferimento</i>	Relazione	125	L	125
<i>Candidatura</i>	Entità	125	L	125
<i>Avanzamento</i>	Relazione	375	L	375
<i>StoricoStati</i>	Entità Debole	375	L	375
<i>Prevede</i>	Relazione	12	L	12
<i>Colloquio</i>	Entità	12	L	12
<i>Genera</i>	Relazione	2	L	2
<i>Offerta</i>	Entità	2	L	2
Totale: 1.039 accessi $\times$ 50/giorno = 51.950 accessi al giorno				

O16 - Modificare il livello di padronanza di una Skill

La navigazione si concentra esclusivamente sulla tabella ponte derivante dalla relazione N:M. Viene effettuato un accesso diretto in modifica (UPDATE) sul costrutto *Competenza* per alterare il valore del suo attributo intrinseco (*Livello*).



Concetto	Costrutto	Accessi	Tipo	Costo parziale
<i>Competenza</i>	Relazione	1	L	1
<i>Competenza</i>	Relazione	1	S	2
Totale: 3 accessi $\times$ 25/giorno = 75 accessi al giorno				

3.4 Raffinamento dello schema

Prima di procedere con la traduzione nel modello relazionale, è necessario risolvere i costrutti del modello concettuale E/R che non trovano un corrispettivo diretto nelle logiche dei database SQL.

Analisi delle ridondanze

Durante la fase di raffinamento, è stata valutata l'introduzione di attributi derivati (ridondanze) al fine di ottimizzare le letture, come ad esempio un contatore del "Numero di Candidature" per ogni Utente o "Numero di Posizioni Aperte" per le Aziende. Analizzando tuttavia il carico applicativo (Tabelle degli accessi), si evince che le operazioni di inserimento e aggiornamento in queste aree (es. O5, O7) hanno frequenze giornaliere molto elevate. L'introduzione di dati ridondanti avrebbe imposto un aggiornamento continuo dei contatori, aumentando drasticamente il rischio di inconsistenze e i costi di scrittura. Pertanto, si è scelto di mantenere lo schema normalizzato e calcolare tali valori dinamicamente a runtime tramite funzioni di aggregazione SQL (COUNT).

Eliminazione delle gerarchie

Nello schema concettuale è presente una generalizzazione totale ed esclusiva (T, E) tra l'entità padre *Utente* e le entità figlie *Candidato* e *Amministratore*. Si è scelto di risolvere questa gerarchia adottando la strategia del **collasso verso il basso** (partizionamento orizzontale), eliminando del tutto l'entità padre e replicando i suoi attributi (*NomeUtente*, *Password*, *Nome*, *Cognome*, *Email*) in entrambe le entità figlie.

Sebbene l'assenza di attributi specifici nelle entità figlie rendesse tecnicamente percorribile anche l'opzione del collasso verso l'alto (creando un'unica tabella *Utente* dotata di un attributo "Ruolo", senza generare alcun valore NULL), questa strada è stata categoricamente scartata per garantire una ferrea **integrità referenziale**. Mantenendo due tabelle fisicamente separate, si forza il database a livello strutturale a impedire anomalie logiche o di business: diventa infatti matematicamente impossibile per un *Amministratore* possedere una *Skill* o inviare una *Candidatura*, poiché i vincoli di chiave esterna (Foreign Key) di tali operazioni punteranno unicamente alla tabella *Candidato*.

Eliminazione di attributi composti e multivalore

Nello schema E/R realizzato non sono presenti attributi composti o attributi multivalore, pertanto non si è resa necessaria alcuna ristrutturazione in tal senso. Tutte le proprietà delle entità godono già di atomicità.

Scelta delle chiavi primarie e gestione delle entità deboli

Per la quasi totalità delle entità, l'identificatore principale scelto in fase concettuale è stato promosso a chiave primaria della rispettiva relazione (es. *ID_Azienda*, *ID_Posizione*, *ID_Skill*).

Per le nuove tabelle *Candidato* e *Amministratore*, derivanti dal collasso della gerarchia, si è deciso di **introdurre un codice identificativo surrogato** (*ID_Candidato* e *ID_Amministratore*, di tipo intero autoincrementante) da utilizzare come chiave primaria,

declassando l'attributo *NomeUtente*. Questa scelta architetturale è dettata da ragioni di ottimizzazione prestazionale: utilizzare una stringa di testo (come un *NomeUtente*) come chiave primaria avrebbe appesantito enormemente le operazioni di JOIN e sprecato memoria, considerando che tale chiave dovrà essere replicata come Foreign Key in centinaia di migliaia di record nelle tabelle *Candidatura* e *Competenza*.

Infine, per quanto riguarda l'entità debole *StoricoStati*, la sua traduzione logica richiede rigorosamente l'utilizzo di una **chiave primaria composta**. Dipendendo essenzialmente da *Candidatura*, la sua Primary Key sarà formata dall'unione della chiave esterna dell'entità forte (*ID_Candidatura*) e dal proprio attributo discriminante parziale (*Data_Cambio*), garantendo così l'univocità temporale di ogni singolo salto di stato per una specifica application.

3.5 Traduzione di entità e associazioni in relazioni

La transizione dal modello concettuale a quello relazionale è stata condotta applicando rigorosamente le regole standard di derivazione:

- **Associazioni 1:N:** Sono state tradotte introducendo la chiave primaria dell'entità forte (lato 1) come chiave esterna (Foreign Key) all'interno dell'entità lato N. Questo approccio è stato utilizzato per le relazioni *Proposta*, *Sottomissione*, *Riferimento*, *Prevede* e *Genera*.
- **Associazioni N:M:** Le relazioni *Competenza* e *Requisito* sono state reificate, diventando a tutti gli effetti delle nuove tabelle. La loro chiave primaria è composta dall'unione delle chiavi esterne delle entità collegate.
- **Entità Deboli:** L'entità *StoricoStati* è stata tradotta in una tabella la cui chiave primaria composta è formata dalla chiave esterna dell'entità da cui dipende (*ID_Candidatura*) e dal proprio attributo parziale (*Data_Cambio*).
- **Gerarchie:** Come precedentemente analizzato, l'entità *Utente* è stata omessa in favore delle tabelle *Candidato* e *Amministratore*.

3.6 Schema relazionale finale

Di seguito viene presentato lo schema logico definitivo. Per convenzione notazionale, le **Chiavi Primarie** (Primary Key) sono sottolineate, mentre le **Chiavi Esterne** (Foreign Key) sono contrassegnate da un asterisco (*).

Anagrafiche e Utenze

- **Candidato**(ID_Candidato, NomeUtente, Password, Nome, Cognome, Email)
- **Amministratore**(ID_Amministratore, NomeUtente, Password, Nome, Cognome, Email)
- **Azienda**(ID_Azienda, Nome_Azienda, Settore, Sede_Principale)

*Nota: Sulle tabelle Candidato e Amministratore verranno imposti vincoli di **UNIQUE** sugli attributi Email e NomeUtente.*

Core Business: Annunci e Skill

- **PosizioneLavorativa**(ID_Posizione, Titolo, Modalita, Range_RAL, Data_Scadenza_Annuncio, ID_Azienda*)
- **Skill**(ID_Skill, Nome_Skill)
- **Requisito**(ID_Posizione*, ID_Skill*, Livello_Minimo)
- **Competenza**(ID_Candidato*, ID_Skill*, Livello)

Processo di Selezione: Candidature, Colloqui e Offerte

- **Candidatura**(ID_Candidatura, Data_Creazione, ID_Candidato*, ID_Posizione*)
- **StoricoStati**(ID_Candidatura*, Data_Cambio, Stato)
- **Colloquio**(ID_Colloquio, Data_Ora, Tipo_Colloquio, Esito, Recruiter, Note, ID_Candidatura*)
- **Offerta**(ID_Offerta, RAL_Proposta, Benefit, Data_Scadenza_Offerta, Stato_Offerta, ID_Candidatura*)

*Nota metodologica: In fase di traduzione delle relazioni N:M **Competenza** e **Requisito**, sono stati esplicitati e integrati gli attributi **Livello** e **Livello_Minimo**. Pur non essendo stati formalizzati graficamente nel diagramma E/R di partenza, la loro presenza è stata ritenuta strutturalmente indispensabile per soddisfare i requisiti emersi durante la fase di intervista (Capitolo 1).*

3.7 Schema relazionale visivo

Per una maggiore chiarezza espositiva e per facilitare la successiva fase di implementazione fisica, si riporta di seguito la rappresentazione grafica dello schema logico relazionale, con evidenza delle dipendenze funzionali (Foreign Key) che legano le tabelle del sistema.

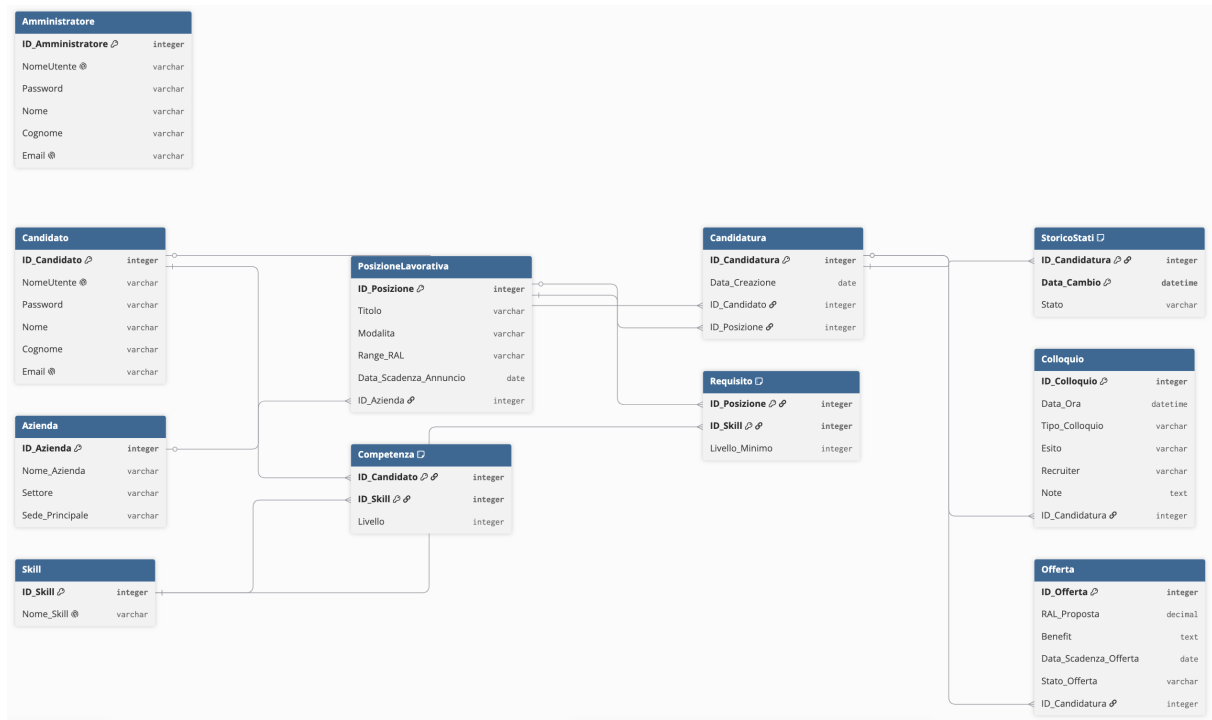


Figura 7: Rappresentazione visiva dello Schema Relazionale definitivo.

4 Implementazione Fisica

In questa fase, lo schema logico relazionale definitivo viene tradotto nel linguaggio DDL (Data Definition Language) per la creazione fisica della base di dati sull'engine **MySQL**.

Al fine di ottimizzare il carico computazionale e prevenire i colli di bottiglia emergenti durante l'analisi degli accessi, lo script include la definizione di vincoli di integrità ferrei e di indici secondari non clusterizzati sulle chiavi esterne a maggior frequenza di interrogazione.

4.1 Definizione dello Schema (DDL)

Di seguito viene riportato lo script SQL completo per la generazione dell'infrastruttura di persistenza, comprensivo dei costrutti per la risoluzione della gerarchia utenze, la creazione delle tabelle ponte e la definizione degli indici secondari.

```
CREATE DATABASE IF NOT EXISTS CareerZen;
USE CareerZen;

CREATE TABLE Candidato (
    ID_Candidato INT AUTO_INCREMENT PRIMARY KEY,
    NomeUtente VARCHAR(50) UNIQUE NOT NULL,
    Password VARCHAR(255) NOT NULL,
    Nome VARCHAR(50) NOT NULL,
    Cognome VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Amministratore (
    ID_Ammministratore INT AUTO_INCREMENT PRIMARY KEY,
    NomeUtente VARCHAR(50) UNIQUE NOT NULL,
    Password VARCHAR(255) NOT NULL,
    Nome VARCHAR(50) NOT NULL,
    Cognome VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Azienda (
    ID_Azienda INT AUTO_INCREMENT PRIMARY KEY,
    Nome_Azienda VARCHAR(100) NOT NULL,
    Settore VARCHAR(100) NOT NULL,
    Sede_Principale VARCHAR(100) NOT NULL
);

CREATE TABLE Skill (
    ID_Skill INT AUTO_INCREMENT PRIMARY KEY,
    Nome_Skill VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE PosizioneLavorativa (
    ID_Posizione INT AUTO_INCREMENT PRIMARY KEY,
    Titolo VARCHAR(100) NOT NULL,
    Modalita VARCHAR(50) NOT NULL,
    Range_RAL VARCHAR(50),
    Data_Scadenza_Annuncio DATE NOT NULL,
    ID_Azienda INT NOT NULL,
    CONSTRAINT fk_pos_azienza FOREIGN KEY (ID_Azienda)
        REFERENCES Azienda(ID_Azienda) ON DELETE CASCADE
);

CREATE TABLE Competenza (
    ID_Candidato INT,
    ID_Skill INT,
    Livello INT NOT NULL CHECK (Livello BETWEEN 1 AND 5),
    PRIMARY KEY (ID_Candidato, ID_Skill),
```

```

        CONSTRAINT fk_comp_cand FOREIGN KEY (ID_Candidato)
            REFERENCES Candidato(ID_Candidato) ON DELETE CASCADE,
        CONSTRAINT fk_comp_skill FOREIGN KEY (ID_Skill)
            REFERENCES Skill(ID_Skill) ON DELETE RESTRICT
    );

CREATE TABLE Requisito (
    ID_Posizione INT,
    ID_Skill INT,
    Livello_Minimo INT NOT NULL CHECK (Livello_Minimo BETWEEN 1 AND 5),
    PRIMARY KEY (ID_Posizione, ID_Skill),
    CONSTRAINT fk_req_pos FOREIGN KEY (ID_Posizione)
        REFERENCES PosizioneLavorativa(ID_Posizione) ON DELETE CASCADE,
    CONSTRAINT fk_req_skill FOREIGN KEY (ID_Skill)
        REFERENCES Skill(ID_Skill) ON DELETE RESTRICT
);

CREATE TABLE Candidatura (
    ID_Candidatura INT AUTO_INCREMENT PRIMARY KEY,
    Data_Creazione DATE NOT NULL,
    ID_Candidato INT NOT NULL,
    ID_Posizione INT NOT NULL,
    CONSTRAINT fk_cand_candidato FOREIGN KEY (ID_Candidato)
        REFERENCES Candidato(ID_Candidato) ON DELETE CASCADE,
    CONSTRAINT fk_cand_posizione FOREIGN KEY (ID_Posizione)
        REFERENCES PosizioneLavorativa(ID_Posizione) ON DELETE CASCADE
);

CREATE TABLE StoricoStati (
    ID_Candidatura INT,
    Data_Cambio DATETIME,
    Stato VARCHAR(50) NOT NULL,
    PRIMARY KEY (ID_Candidatura, Data_Cambio),
    CONSTRAINT fk_storico_cand FOREIGN KEY (ID_Candidatura)
        REFERENCES Candidatura(ID_Candidatura) ON DELETE CASCADE
);

CREATE TABLE Colloquio (
    ID_Colloquio INT AUTO_INCREMENT PRIMARY KEY,
    Data_Ora DATETIME NOT NULL,
    Tipo_Colloquio VARCHAR(50) NOT NULL,
    Esito VARCHAR(50),
    Recruiter VARCHAR(100),
    Note TEXT,
    ID_Candidatura INT NOT NULL,
    CONSTRAINT fk_coll_cand FOREIGN KEY (ID_Candidatura)
        REFERENCES Candidatura(ID_Candidatura) ON DELETE CASCADE
);

CREATE TABLE Offerta (
    ID_Offerta INT AUTO_INCREMENT PRIMARY KEY,
    RAL_Proposta DECIMAL(10,2) NOT NULL CHECK (RAL_Proposta > 0),
    Benefit TEXT,
    Data_Scadenza_Offerta DATE NOT NULL,
    Stato_Offerta VARCHAR(50) NOT NULL,
    ID_Candidatura INT NOT NULL,
    CONSTRAINT fk_off_cand FOREIGN KEY (ID_Candidatura)
        REFERENCES Candidatura(ID_Candidatura) ON DELETE CASCADE
);

CREATE INDEX idx_candidatura_candidato ON Candidatura(ID_Candidato);
CREATE INDEX idx_candidatura_posizione ON Candidatura(ID_Posizione);
CREATE INDEX idx_posizione_azienza ON PosizioneLavorativa(ID_Azienda);
CREATE INDEX idx_storico_stato ON StoricoStati(Stato);

```

4.2 Traduzione delle operazioni in query SQL (DML)

In questa sezione vengono formalizzate le interrogazioni SQL relative alle operazioni previste dal sistema. Il codice è stato redatto prevedendo l'impiego di parametri dinamici (indicati dal placeholder ?), predisponendo di fatto le query per l'incapsulamento all'interno dei PreparedStatement dell'applicativo Java (JDBC).

O1 - Iscrivere un nuovo Utente (Candidato o Amministratore)

A seguito del collasso della gerarchia *Utente* nello schema logico, l'operazione si traduce in una singola INSERT direzionata, a livello applicativo, verso la tabella corrispondente al ruolo scelto. Si riportano entrambe le casistiche.

```
INSERT INTO Candidato (NomeUtente, Password, Nome, Cognome, Email)
VALUES (?, ?, ?, ?, ?);

INSERT INTO Amministratore (NomeUtente, Password, Nome, Cognome,
Email)
VALUES (?, ?, ?, ?, ?);
```

O2 - Registrare una nuova Azienda nel sistema

Si tratta di una singola operazione atomica di inserimento in anagrafica, senza dipendenze strutturali esterne.

```
INSERT INTO Azienda (Nome_Azienda, Settore, Sede_Principale)
VALUES (?, ?, ?);
```

O3 - Pubblicare una nuova Posizione Lavorativa con le relative Skill

L'operazione coinvolge due tabelle e viene isolata all'interno di una transazione. Viene dapprima generata la *PosizioneLavorativa* legata all'Azienda, dopodiché l'ID generato dinamicamente viene recuperato per inserire i molteplici record nella tabella ponte *Requisito*.

```
START TRANSACTION;

INSERT INTO PosizioneLavorativa (Titolo, Modalita, Range_RAL,
Data_Scadenza_Annuncio, ID_Azienda)
VALUES (?, ?, ?, ?, ?);

SET @nuova_posizione_id = LAST_INSERT_ID();

INSERT INTO Requisito (ID_Posizione, ID_Skill, Livello_Minimo)
VALUES (@nuova_posizione_id, ?, ?);

COMMIT;
```

O4 - Associare una nuova Skill al profilo di un Utente

Operazione di scrittura diretta sulla tabella ponte *Competenza* per istanziare il legame N:M tra il Candidato e il catalogo Skill, specificandone la padronanza.

```
INSERT INTO Competenza (ID_Candidato, ID_Skill, Livello)
VALUES (?, ?, ?);
```

O5 - Registrare l'invio di una nuova Candidatura

L'operazione richiede l'inserimento simultaneo di dati in due tabelle distinte (*Candidatura* e *StoricoStati*). Per garantire l'atomicità e prevenire inconsistenze (es. la creazione di una candidatura orfana del suo stato iniziale), le istruzioni vengono inglobate in una transazione esplicita. L'identificativo generato dalla prima operazione viene recuperato dinamicamente per popolare la chiave esterna della seconda.

```
START TRANSACTION;

INSERT INTO Candidatura (Data_Creazione, ID_Candidato,
    ID_Posizione)
VALUES (CURRENT_DATE(), ?, ?);

SET @nuova_candidatura_id = LAST_INSERT_ID();

INSERT INTO StoricoStati (ID_Candidatura, Data_Cambio, Stato)
VALUES (@nuova_candidatura_id, CURRENT_TIMESTAMP(), 'Inviata');

COMMIT;
```

O6 - Inserire un nuovo step di Colloquio

Operazione di scrittura diretta che istanzia un nuovo record relativo a un round di selezione, legandolo tramite chiave esterna alla candidatura di riferimento.

```
INSERT INTO Colloquio (Data_Ora, Tipo_Colloquio, Esito, Recruiter
    , Note, ID_Candidatura)
VALUES (?, ?, ?, ?, ?, ?);
```

O7 - Registrare un cambio di stato per una Candidatura

Aggiunta in logica *append-only* di una nuova transizione temporale per una candidatura esistente, sfruttando il timestamp corrente del DBMS per garantire l'univocità della chiave primaria composta.

```
INSERT INTO StoricoStati (ID_Candidatura, Data_Cambio, Stato)
VALUES (?, CURRENT_TIMESTAMP(), ?);
```

O8 - Memorizzare una nuova Offerta economica ricevuta

Inserimento dei dettagli di un'offerta contrattuale generata al termine dell'iter selettivo. Il DBMS verificherà in automatico il vincolo CHECK sulla positività della RAL proposta.

```
INSERT INTO Offerta (RAL_Proposta, Benefit, Data_Scadenza_Offerta
    , Stato_Offerta, ID_Candidatura)
VALUES (?, ?, ?, ?, ?);
```


O9 - Visualizzare lo storico completo di una Candidatura

Al fine di evitare un oneroso e inefficiente prodotto cartesiano (derivante dal JOIN simultaneo di tre relazioni 1:N indipendenti rispetto alla stessa candidatura), l'estrazione radiale dei dati viene scomposta in tre **SELECT** mirate. Questo approccio riflette le best practice di interrogazione tramite layer applicativo (es. JDBC).

```
SELECT Data_Cambio, Stato
FROM StoricoStati
WHERE ID_Candidatura = ?
ORDER BY Data_Cambio DESC;

SELECT Data_Ora, Tipo_Colloquio, Esito, Recruiter, Note
FROM Colloquio
WHERE ID_Candidatura = ?
ORDER BY Data_Ora ASC;

SELECT RAL_Proposta, Benefit, Data_Scadenza_Offerta,
       Stato_Offerta
FROM Offerta
WHERE ID_Candidatura = ?;
```

O10 - Ricercare Posizioni Lavorative filtrando per compatibilità

Query di interrogazione avanzata che sfrutta le tabelle ponte per incrociare la domanda e l'offerta. Tramite un costrutto di JOIN multiplo, vengono filtrate esclusivamente le posizioni lavorative per le quali il candidato possiede una determinata *Skill* con un *Livello* maggiore o uguale al *Livello_Minimo* richiesto dall'azienda.

```
SELECT DISTINCT
    pl.ID_Posizione,
    pl.Titolo,
    a.Nome_Azienda,
    pl.Modalita,
    pl.Range_RAL
FROM PosizioneLavorativa pl
JOIN Azienda a ON pl.ID_Azienda = a.ID_Azienda
JOIN Requisito r ON pl.ID_Posizione = r.ID_Posizione
JOIN Competenza c ON r.ID_Skill = c.ID_Skill
WHERE c.ID_Candidato = ?
      AND c.Livello >= r.Livello_Minimo;
```

O11 - Segnalare e bloccare un'Azienda

In assenza di un attributo di partizionamento logico (soft-delete) nell'anagrafica, l'estromissione dell'azienda si traduce in un'eliminazione fisica. Grazie alle politiche di reazione **ON DELETE CASCADE** definite in sede di DDL, questa singola operazione garantisce la rimozione a cascata di tutti gli annunci (*PosizioneLavorativa*) e delle dipendenze associate a tale impresa, mantenendo la base dati consistente.

```
DELETE FROM Azienda
WHERE ID_Azienda = ?;
```

O12 - Aggiornare l'esito e le note di un Colloquio

Semplice operazione di UPDATE puntuale finalizzata a sovrascrivere i campi testuali e di valutazione a seguito dello svolgimento di uno step di selezione.

```
UPDATE Colloquio
SET Esito = ?, Note = ?
WHERE ID_Colloquio = ?;
```

O13 - Registrare la decisione finale su un'Offerta economica

Modifica dello stato di una specifica proposta contrattuale (es. transizione da "In negoziazione" ad "Accettata" o "Rifiutata").

```
UPDATE Offerta
SET Stato_Offerta = ?
WHERE ID_Offerta = ?;
```

O14 - Visualizzare la dashboard principale del Candidato

Questa query estrae l'elenco delle candidature attive di uno specifico utente, affiancando i dettagli dell'annuncio e dell'azienda. La complessità logica risiede nell'estrazione dello stato corrente: poiché l'iter genera molteplici record storici, viene utilizzata una subquery correlata sull'entità *StoricoStati* per selezionare esclusivamente la transizione di stato più recente (MAX(Data_Cambio)), nascondendo le fasi obsolete.

```
SELECT
    c.ID_Candidatura,
    pl.Titolo AS Posizione,
    a.Nome_Azienda AS Azienda,
    c.Data_Creazione AS Inviata_Il,
    s.Stato AS Stato_Attuale,
    s.Data_Cambio AS Ultimo_Aggiornamento
FROM Candidatura c
JOIN PosizioneLavorativa pl ON c.ID_Posizione = pl.ID_Posizione
JOIN Azienda a ON pl.ID_Azienda = a.ID_Azienda
JOIN StoricoStati s ON c.ID_Candidatura = s.ID_Candidatura
WHERE c.ID_Candidato = ?
    AND s.Data_Cambio = (
        SELECT MAX(Data_Cambio)
        FROM StoricoStati
        WHERE ID_Candidatura = c.ID_Candidatura
    )
ORDER BY s.Data_Cambio DESC;
```

O15 - Generare un report statistico sulle performance aziendali

Query analitica (Batch) progettata per fornire una metrica completa sull'imbuto di conversione (funnel) del recruiting per una specifica azienda. L'utilizzo combinato di LEFT JOIN e COUNT(DISTINCT) è fondamentale per due ragioni: previene l'alterazione dei conteggi dovuta al prodotto cartesiano delle relazioni N:M e garantisce che l'azienda compaia nel report anche qualora non vi siano ancora candidature o colloqui registrati.

```

SELECT
    a.Nome_Azienda ,
    COUNT(DISTINCT pl.ID_Posizione) AS Posizioni_Aperte ,
    COUNT(DISTINCT c.ID_Candidatura) AS Candidature_Ricevute ,
    COUNT(DISTINCT col.ID_Colloquio) AS Colloqui_Effettuati ,
    COUNT(DISTINCT o.ID_Offerta) AS Offerte_Emesse
FROM Azienda a
LEFT JOIN PosizioneLavorativa pl ON a.ID_Azienda = pl.ID_Azienda
LEFT JOIN Candidatura c ON pl.ID_Posizione = c.ID_Posizione
LEFT JOIN Colloquio col ON c.ID_Candidatura = col.ID_Candidatura
LEFT JOIN Offerta o ON c.ID_Candidatura = o.ID_Candidatura
WHERE a.ID_Azienda = ?
GROUP BY a.ID_Azienda;

```

O16 - Modificare il livello di padronanza di una Skill

Operazione di manutenzione del profilo utente. Consiste in un UPDATE puntuale localizzato esclusivamente sulla tabella ponte *Competenza*, andando ad alterare un attributo intrinseco della relazione senza intaccare le anagrafiche di base.

```

UPDATE Competenza
SET Livello = ?
WHERE ID_Candidato = ? AND ID_Skill = ?;

```

5 Realizzazione dell'Applicazione

A coronamento della progettazione teorica e della creazione fisica della base di dati, è stato sviluppato un applicativo desktop per dimostrare la validità e la robustezza del modello relazionale. L'applicazione funge da livello di presentazione (Front-End) e da strato di logica applicativa, interfacciandosi con il DBMS MySQL tramite il driver JDBC.

5.1 Architettura di Sistema e Pattern DAO

Per garantire un elevato livello di manutenibilità e un basso accoppiamento, il software è stato strutturato secondo il pattern architetturale **DAO (Data Access Object)**. L'intera logica di interazione con il database è stata incapsulata in un'unica classe centralizzata (`CareerZenDAO.java`).

Questa separazione netta tra l'interfaccia utente (View) e il database (Model) assicura che il codice grafico non contenga mai istruzioni SQL dirette. Se in futuro l'applicazione dovesse migrare verso un'architettura Web o Mobile, il livello di persistenza DAO rimarrebbe invariato, agendo come nucleo per eventuali API RESTful.

5.2 Sicurezza, Transazioni e Integrità

L'implementazione a livello di codice non si è limitata alla semplice esecuzione delle query, ma ha tenuto conto delle principali problematiche di un ambiente di produzione:

- **Prevenzione SQL Injection:** Tutte le interrogazioni e le modifiche ai dati (DML) sono state reificate utilizzando i `PreparedStatement` di Java. Questo impedisce l'iniezione di codice malevolo tramite i form dell'interfaccia.
- **Gestione delle Transazioni (ACID):** Per le operazioni complesse che coinvolgono scritture multiple su tabelle diverse, l'auto-commit di JDBC è stato disabilitato (`connection.setAutoCommit(false)`). Durante l'operazione di Invio Candidatura (**O5**), l'inserimento in *Candidatura* e l'inizializzazione in *StoricoStati* avvengono in un unico blocco transazionale atomico. La medesima logica è applicata alla pubblicazione di una posizione con i relativi requisiti (**O3**), isolando il recupero di `LAST_INSERT_ID()` e l'inserimento in *Requisito*. In caso di eccezione, viene eseguito un `rollback()` per mantenere il database in uno stato consistente.
- **Sanitizzazione e Guardrail di Interfaccia:** Per mitigare i rischi legati all'inserimento di dati da parte dell'utente finale, l'integrità referenziale delle chiavi esterne è protetta a livello di interfaccia tramite l'uso estensivo di `JComboBox`. Le tendine per la selezione delle aziende e delle skill caricano dinamicamente gli identificativi validi dal database (tramite i metodi helper `getElencoAziende()` e `getElencoSkill()`), impedendo l'inserimento di chiavi orfane o stringhe non normalizzate.

5.3 Interfaccia Utente (GUI) e Scelte di Design

L'interfaccia grafica è stata sviluppata utilizzando la libreria **Java Swing**. Per discostarsi dal design nativo tridimensionale e garantire la massima leggibilità su ambienti multi-piattaforma, è stato applicato un approccio stilistico *Flat*, caratterizzato da una palette

minimalista, pulsanti opachi a tinta unita e finestre di dialogo modali (JDialog) per isolare i flussi operativi di inserimento.

5.3.1 Accesso e Controllo dei Ruoli

L'applicazione avvia il proprio ciclo di vita con un portale di autenticazione (**O1**). Sfruttando il partizionamento orizzontale adottato in fase di raffinamento logico (tabelle distinte per *Candidato* e *Amministratore*), la logica di login interroga la tabella pertinente in base alla selezione della casella di controllo amministrativa, impedendo a monte anomalie di accesso o escalations di privilegi.

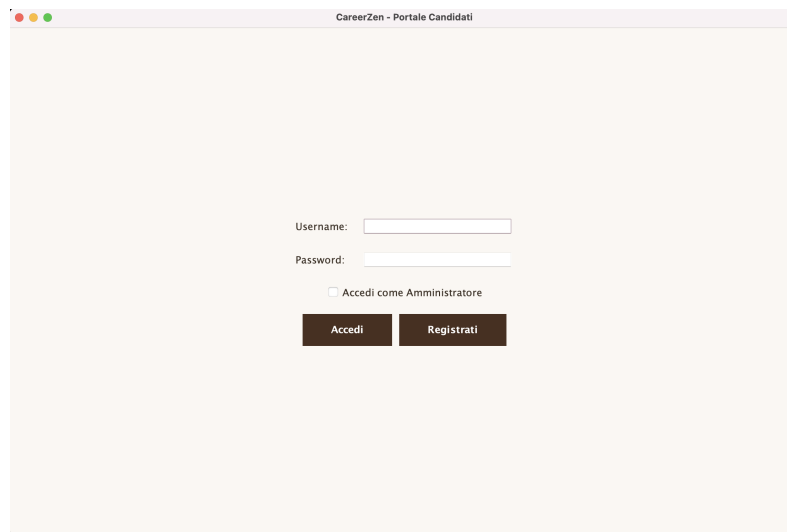


Figura 8: Interfaccia di Login e instradamento basato su ruoli fisici.

5.3.2 Evoluzione in Tracker Collaborativo: Dashboard Candidato

In fase di sviluppo, l'applicazione è stata sottoposta a un'evoluzione logica rispetto ai requisiti preliminari, virando verso un modello di **Tracker Collaborativo (Crowdsourced)**. Al fine di supportare un reale tracciamento personale delle sottomissioni esterne (es. annunci individuati su canali terzi come LinkedIn), il potere di censire una posizione lavorativa con i relativi requisiti (**O3**) è stato assegnato al *Candidato*.

Per preservare l'integrità dell'anagrafica aziendale, il candidato non inserisce il nome dell'azienda a testo libero, ma è vincolato a selezionare un'entità aziendale pre-validata e inserita a sistema dall'Amministratore. La Dashboard del Candidato (**O14**) aggrega ed espone dinamicamente sia il catalogo delle competenze accumulate dal profilo (**O4**, **O16**) sia la timeline dello stato corrente di ciascuna candidatura attiva.

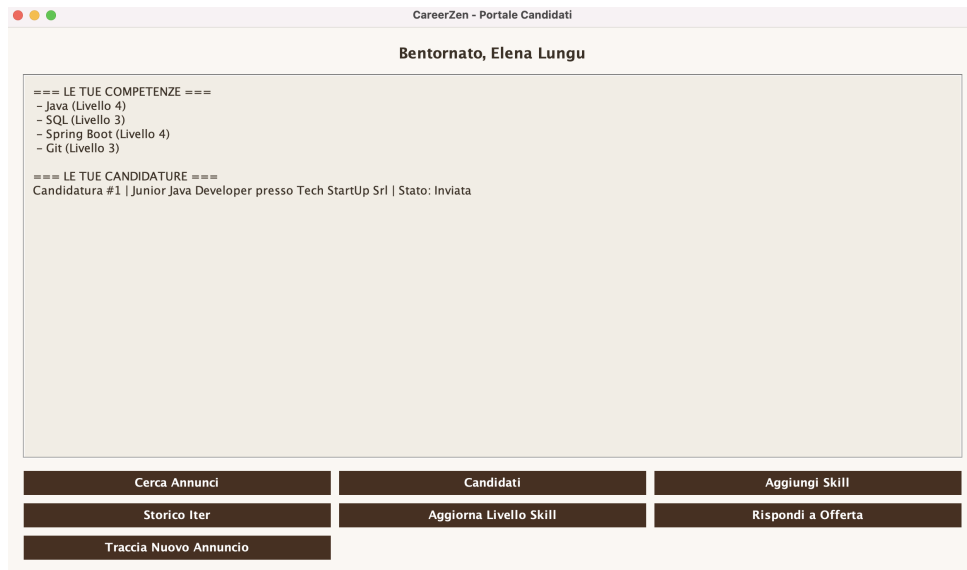


Figura 9: Dashboard del Candidato con riepilogo del profilo e pannello di tracciamento.

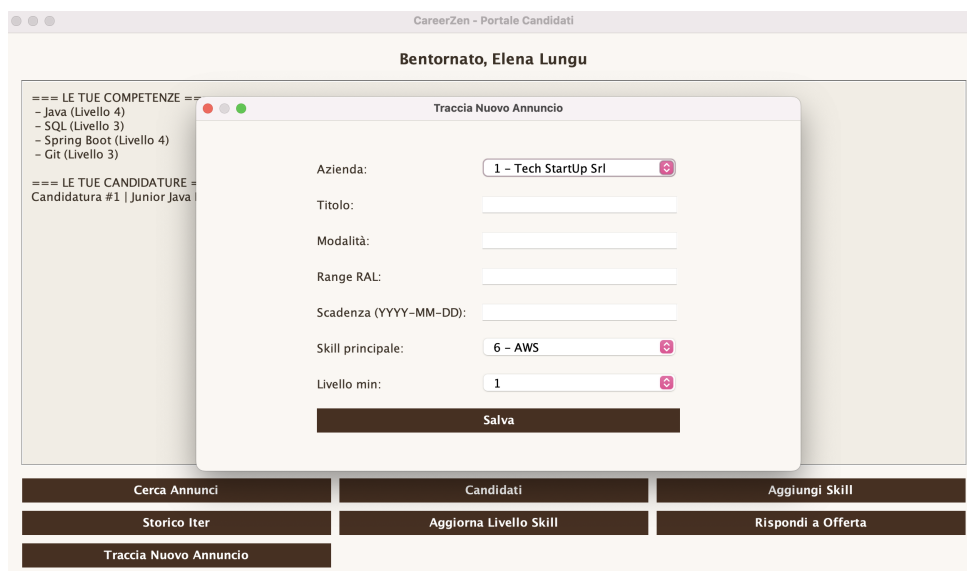


Figura 10: Dialog di inserimento annuncio con vincolo referenziale su Aziende e Skill tramite JComboBox.

5.3.3 Pannello di Controllo dell'Amministratore

A seguito del pivot collaborativo dell'applicazione, il ruolo dell'Amministratore si è concentrato sulle funzioni di alta supervisione e orchestrazione dei processi. L'Admin mantiene l'esclusiva sul data-entry delle realtà societarie partner (**O2**) e sulle funzioni di moderazione e sanzionamento, potendo rimuovere un'azienda dal sistema (**O11**) e innescando la rimozione a cascata (**CASCADE**) delle posizioni e candidature correlate.

Inoltre, l'interfaccia amministrativa funge da pannello di back-office per governare l'avanzamento degli stati di selezione (**O7**), la pianificazione dei colloqui (**O6**), la registrazione dei relativi esiti tecnici (**O12**) e l'emissione delle proposte contrattuali (**O8**),

culminando nella generazione dei report statistici aggregati sulle performance delle aziende (O15).

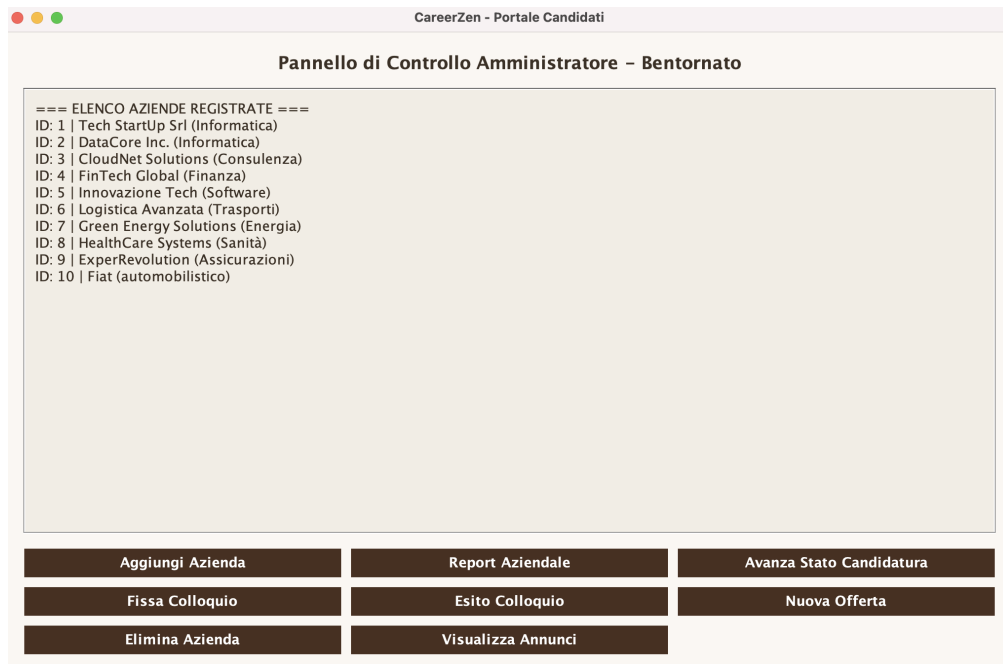


Figura 11: Pannello Amministratore: monitoraggio analitico delle aziende e gestione dei flussi transazionali.

5.4 Sviluppi Futuri

Il sistema, nella sua attuale configurazione, soddisfa esaurientemente l'intero carico di lavoro e i vincoli relazionali del dominio. In un'ottica di industrializzazione del prodotto, sono stati delineati i seguenti pilastri di sviluppo futuro per la scalabilità del software:

1. **Connection Pooling:** Integrazione di un pooler di connessioni di livello enterprise (es. *HikariCP*) nel Data Access Layer per eliminare l'overhead computazionale dovuto all'apertura e chiusura fisica delle connessioni JDBC per ciascuna istanza di query interattiva.
2. **Autenticazione Stateless e Cifratura:** Sostituzione della memorizzazione delle credenziali in chiaro tramite funzioni di hashing asimmetriche dotate di salting (es. *Bcrypt*) per la conformità agli standard di sicurezza dei dati, associata all'impiego di *JSON Web Tokens (JWT)* per disaccoppiare il backend e predisporlo a una scalabilità orizzontale.